
Security Review Report
NM-1028 - Null Solutions



NETHERMIND
SECURITY

(August 25, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Architecture	4
4.1.1	Core	4
4.1.2	Periphery	4
4.2	Life cycle	5
4.2.1	Market Creation	5
4.2.2	User Interaction	5
4.2.3	Settlement	5
4.3	Settlement Mechanics	5
4.3.1	MarketState	5
4.3.2	Settleable Balance	6
4.3.3	Price PnL	6
4.3.4	Funding	6
4.3.5	Circuit breaker	6
4.4	Funding Rate Model	6
4.5	Oracle	6
4.6	Design decisions	6
5	Risk Rating Methodology	7
6	Issues	8
6.1	[Medium] Shares are calculated before fee recomputation on mint, causing depositors to lose a portion of their assets	8
6.2	[Low] _deposit does not revert on zero shares	9
6.3	[Low] deposit flow is cheaper than mint flow, causing protocol to lose portion of fees	10
6.4	[Info] Inconsistent reentrancy guard implementations across contracts	11
6.5	[Info] No mechanism to prevent deployment of new markets using whitelisted account, if compromised	11
7	Documentation Evaluation	12
8	Test Suite Evaluation	13
8.1	Tests Output	13
9	About Nethermind	22

1 Executive Summary

This document presents the results of the security review conducted by **Nethermind Security** for **Null Solutions**'s smart contracts for the NullIV1 Protocol for perpetual derivative markets.

NullIV1 is a derivatives protocol where the system creates isolated markets consisting of two counterparty pools. A Receiver pool and a Payer pool that accepts ERC-20 tokens as collateral. Participants take receiver (long) or payer (short) positions on the funding rate of an underlying asset price. All collateral is held in a single **NullIV1Market** contract and accounted across two ERC-4626 vaults representing the receiver and payer pools.

NullIV1Factory deploys EIP-1167 proxy clones for markets and paired pools keyed by unique collateral, oracle, and funding rate model. As the core settlement engine, **NullIV1Market** holds all collateral tokens and enforces a balance invariant across the receiver pool, payer pool, and protocol fees. On every interaction, the Settlement library reconciles balances, accrues fees, updates PnL, and settles funding transfers.

The periphery uses **NullIV1Router** as a stateless entry point handling user interactions, slippage protection, deadline checks, and collateral wrapping. Pricing data is sourced from **Chainlink** Data Streams oracles.

The audit comprises 1220 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report five points of attention, where one is classified as Medium, two are classified as Low and two are classified as Informational. The issue is summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

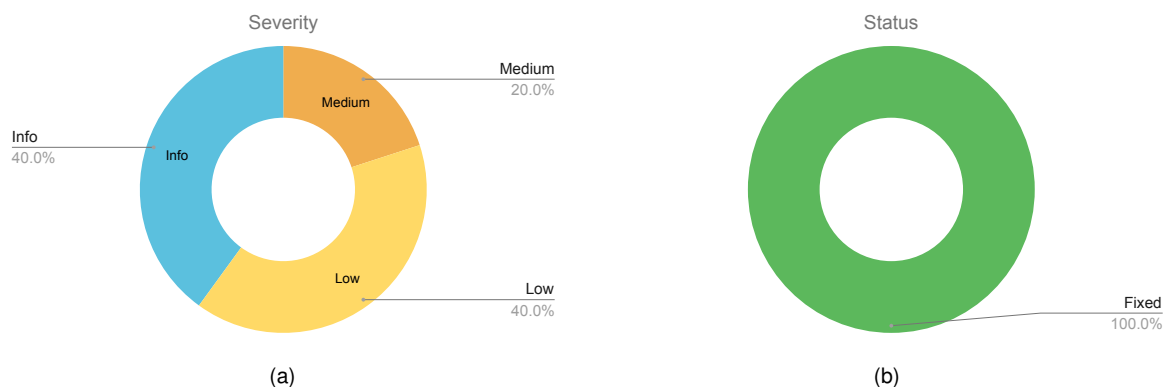


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (1), Low (2), Undetermined (0), Informational (2), Best Practices (0). Distribution of status: Fixed (5), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Aug 18, 2026
Final Report	Aug 25, 2026
Initial Commit	98887f0
Final Commit	1f0562a
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	periphery/NullV1Router.sol	132	55	41.7%	15	202
2	periphery/frm/NullV1ImbalanceFundingRateModel.sol	40	30	75.0%	14	84
3	periphery/oracle/INullV1UpdatableOracle.sol	7	10	142.9%	4	21
4	periphery/oracle/chainlink/NullV1ChainlinkOracleV11.sol	174	108	62.1%	43	325
5	periphery/oracle/chainlink/NullV1ChainlinkOracleFactory.sol	49	16	32.7%	11	76
6	core/NullV1Pool.sol	187	56	29.9%	39	282
7	core/NullV1Factory.sol	125	27	21.6%	21	173
8	core/NullV1NoDelegate.sol	15	9	60.0%	9	33
9	core/NullV1Market.sol	183	77	42.1%	39	299
10	core/NullV1ReentrancyGuard.sol	25	35	140.0%	12	72
11	core/interfaces/INullV1Pool.sol	11	5	45.5%	5	21
12	core/interfaces/INullV1Market.sol	38	30	78.9%	17	85
13	core/interfaces/INullV1FundingRateModel.sol	17	11	64.7%	4	32
14	core/interfaces/INullV1Oracle.sol	9	19	211.1%	6	34
15	core/interfaces/INullV1Factory.sol	26	27	103.8%	14	67
16	core/libraries/ConstantsLib.sol	12	22	183.3%	11	45
17	core/libraries/SettlementLib.sol	148	35	23.6%	22	205
18	core/libraries/ErrorsLib.sol	22	23	104.5%	19	64
	Total	1220	595	48.8%	305	2120

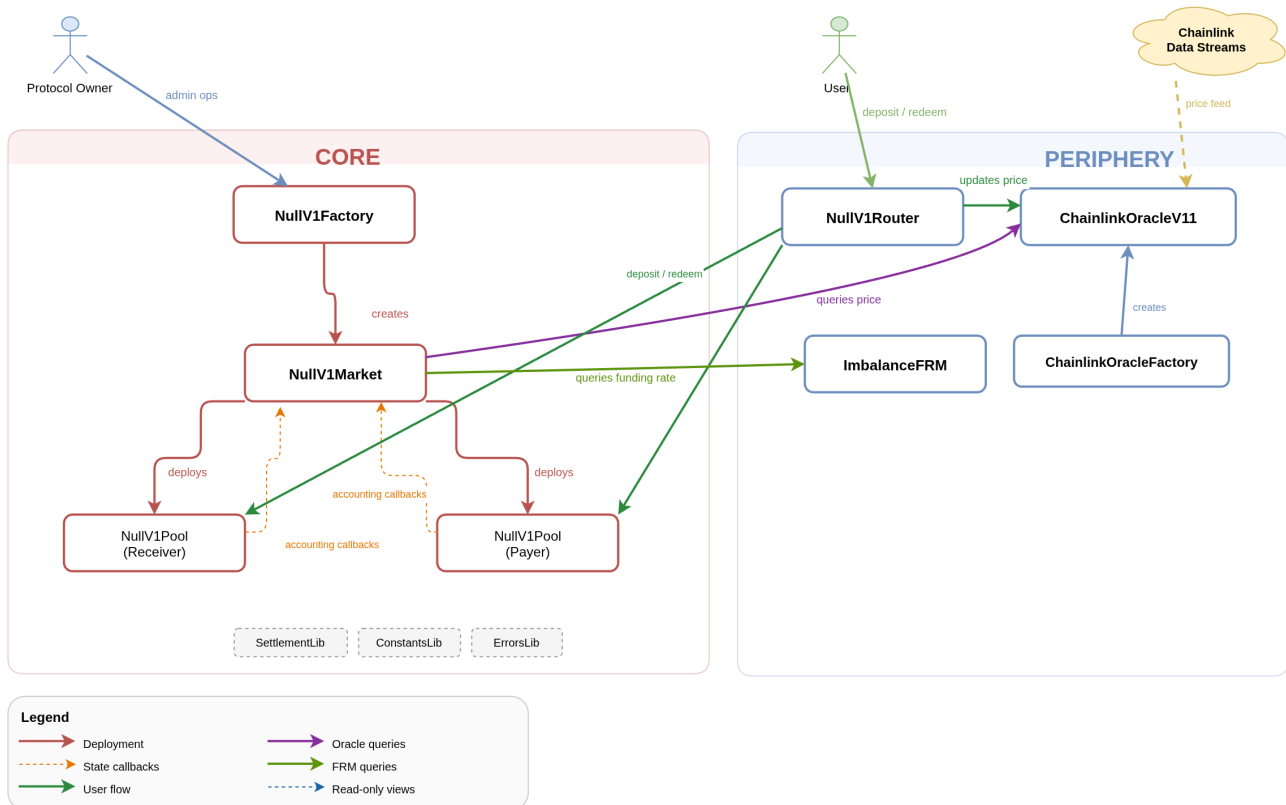
3 Summary of Issues

	Finding	Severity	Update
1	Shares are calculated before fee recomputation on mint , causing depositors to lose a portion of their assets	Medium	Fixed
2	_deposit does not revert on zero shares	Low	Fixed
3	deposit flow is cheaper than mint flow, causing protocol to lose portion of fees	Low	Fixed
4	Inconsistent reentrancy guard implementations across contracts	Info	Fixed
5	No mechanism to prevent deployment of new markets using whitelisted account, if compromised	Info	Fixed

4 System Overview

NullV1 is a decentralised financial protocol designed to create and settle perpetual derivative markets backed by counterparty ERC-4626 liquidity pools. The system creates isolated markets consisting of two counterparty pools: a Receiver pool and a Payer pool. Participants supply ERC-20 as collateral to either side to take exposure, earning returns or incurring losses based on asset price changes provided by oracles and funding rates calculated from pool asset imbalances.

4.1 Architecture



The system is organised into two subsystems, namely Core and Periphery.

4.1.1 Core

The Core smart contracts comprise:

- **NullV1Factory:** Deploys markets via minimal proxy clones. Manages protocol-wide access control (whitelisting, pause, deposit blocking) and FRM enablement. Owned by the protocol admin.
- **NullV1Market:** Central settlement engine for a single market. Holds the MarketState, orchestrates settlement on every interaction, applies fees, and queries the oracle and funding rate. Every market has its own Receiver and Payer pool clones on initialisation.
- **NullV1Pool:** ERC-4626 vault (two instances per market: Receiver and Payer). Users deposit collateral to receive non-transferable shares. On every deposit/withdraw, the pool triggers market settlement before executing the ERC-4626 operation.
- **SettlementLib:** A library containing the core settlement logic for balance reconciliation, collateral fee accrual, price PnL computation, and funding application.
- **ConstantsLib:** Protocol-wide constants
- **ErrorsLib:** Custom error definitions

4.1.2 Periphery

The Periphery smart contracts comprise:

- **NullV1Router:** User-facing entry point for deposits and redemptions. Provides slippage protection, deadline enforcement, and automatic ERC-4626 collateral wrapping/unwrapping.

- **NullV1ImbalanceFRM**: Stateless funding rate model that computes a signed rate based on pool imbalance.
- **NullV1ChainlinkOracleV11**: Price oracle backed by Chainlink Data Streams with multi-session support, staleness checks, and a 15-day stale price escape hatch.
- **NullV1ChainlinkOracleFactory**: Deploys oracle clones via minimal proxy pattern.

4.2 Life cycle

4.2.1 Market Creation

- Protocol owner calls createMarket function.
- Factory validates the FRM is enabled, and the market does not already exist.
- Factory creates market by cloning the NullV1Market implementation and initialises it.
- Market creates two pool instances, one each for Receiver and Payer, reads the initial oracle price, and calls the FRM to initialise any stateful models.

4.2.2 User Interaction

- User calls depositMarket or redeemMarket() via Router contract.
- Router optionally updates the oracle with fresh Chainlink report data.
- Router checks the deadline has not passed.
- For each pool the user is interacting with:
 - Deposit: Router pulls collateral from the user (wrapping into ERC-4626 if needed), deposits into the pool, and verifies shares received meet the minimum slippage bound.
 - Redeem: Router redeems shares from the pool, unwraps collateral if needed, and verifies assets received meet the minimum slippage bound.

4.2.3 Settlement

Every pool operation (deposit, mint, withdraw, redeem) triggers settlement.

- Reentrancy guard acquired : (transient storage, EIP-1153).
- Balance reconciliation: Actual collateral balance is reconciled against accounting. Surpluses are distributed pro-rata to pools; deficits are deducted pro-rata.
- Collateral fee accrual: Annualised fee charged on pool assets and is credited to protocol fees, but is currently disabled.
- Oracle price fetch: Market reads the latest price from the Chainlink oracle.
- Funding rate computation: Market queries the FRM with the current state.
- Price PnL and funding application:
 - Price PnL transfers value between pools proportional to the price change and the receiver's settleable notional.
 - Funding transfers value based on the signed funding rate, elapsed time, and settleable balance. - Both transfers are clamped so neither pool falls below minimum assets.
- Circuit breaker: If the price move exceeds 20%, PnL is skipped, and the reference price jumps to the new level.
- Reentrancy guard released.

4.3 Settlement Mechanics

4.3.1 MarketState

```
MarketState {
    lastUpdate:    uint256 // Timestamp of last settlement
    receiverPool:  uint256 // Gross receiver pool assets (includes MIN_POOL_ASSETS floor)
    payerPool:     uint256 // Gross payer pool assets (includes MIN_POOL_ASSETS floor)
    protocolFees:  uint256 // Accumulated protocol fees
    price:         uint256 // Last settled oracle price
}
```

4.3.2 Settleable Balance

Settlement only applies to the portion of each pool above the minimum asset pool floor. This floor prevents zero-size settlement and ensures a minimum dust balance remains for accounting integrity.

4.3.3 Price PnL

- Positive PnL (price up): transferred from payer to receiver.
- Negative PnL (price down): transferred from receiver to payer.
- Transfer clamped to the losing pool's settleable balance.

4.3.4 Funding

- Positive funding rate: receivers pay payers (reduces net transfer to receiver).
- Negative funding rate: payers pay receivers (increases net transfer to receiver).
- Combined with price PnL into a single net transfer before clamping.

4.3.5 Circuit breaker

If the price difference is greater than 20%, settlement skips PnL entirely and resets the reference price. This prevents exploitation from oracle discontinuities.

4.4 Funding Rate Model

$$f = \mu * (c * R - P) / (c * R + P)$$

Where:

- μ (sensitivity): Maximum absolute funding rate, WAD-scaled.
- c (targetCoverage): Payer-to-receiver ratio at equilibrium, WAD-scaled.
- R = receiver pool balance
- P = payer pool balance

4.5 Oracle

Pricing is sourced from the Chainlink oracle. The protocol supports 3 feed types.

- Regular feed
- Extended feed
- Overnight feed

4.6 Design decisions

- Minimal proxy (Clones) pattern: Markets, pools, and oracles are all deployed as EIP-1167 clones to minimise deployment costs.
- Non-transferable shares: Pool shares cannot be transferred (transfer/transferFrom return false), making positions non-transferable and simplifying the accounting model.
- Interaction-driven settlement: State is only updated when someone interacts with the market.
- Stateless router: The router holds no state, cannot receive ETH, and never custodies funds beyond a single transaction.
- ERC-4626 collateral wrapping: The router transparently handles collateral that is itself an ERC-4626 vault, pulling the underlying asset from users and wrapping it before deposit.
- Settlement logics and calculations are based on the receiver assets.
- PnL and funding are sized off the receiver notional, so a small payer pool's upside on a favorable move — and its funding accrual, which requires no price move at all — is counterparty-sized while its downside is bounded by its own settleable balance.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] Shares are calculated before fee recomputation on mint, causing depositors to lose a portion of their assets

File(s): /src/core/NullV1Pool.sol

Description: NullV1Pool charges its entry and exit fees on notional — the size of the bet a deposit represents — rather than on the deposit amount itself.

The fee is computed by one of two helpers, depending on the flow:

- feeOnTotal — the caller handed over a fixed sum of collateral, so the fee has to be taken out of it;
- feeOnRawNotional — the caller asked for a fixed position size(shares), so the fee has to be added on top;

```

1  function feeOnTotal(uint256 assets, uint256 feeBasisPoints) internal pure returns (uint256) {
2      return Math.mulDiv(assets, feeBasisPoints, feeBasisPoints + MAX_BASIS_POINTS, Math.Rounding.Ceil);
3  }
4
5  function feeOnRawNotional(uint256 notional, uint256 assets, uint256 feeBasisPoints)
6      internal
7      pure
8      returns (uint256)
9  {
10     if (notional == 0 || feeBasisPoints == 0) return 0;
11     uint256 denom = assets * (feeBasisPoints + MAX_BASIS_POINTS);
12     if (notional * feeBasisPoints >= denom) revert ErrorsLib.FeeExceedsAssets();
13     return Math.mulDiv(notional, assets * feeBasisPoints, denom - notional * feeBasisPoints, Math.Rounding.Ceil);
14 }

```

The two functions are meant to be mirror implementations: feeOnRawNotional quotes the fee the user will pay, and feeOnTotal should pull exactly that same fee back out. This equivalence holds only if the fee rate stays constant regardless of deposit size

_notional places the depositor's own asset amount into the denominator:

```

1  function _notional(uint256 assets, bool depositing) private view returns (uint256) {
2      (, uint256 receiverAssets, uint256 payerAssets,) = _market.state();
3      uint256 ownAssets =
4          (poolKind == NullV1PoolKind.Receiver ? receiverAssets : payerAssets) + (depositing ? assets : 0);
5      if (depositing && poolKind == NullV1PoolKind.Receiver) receiverAssets += assets;
6      uint256 settleableR = SettlementLib._settleableAssets(receiverAssets);
7      return ownAssets == 0 || settleableR == 0 || (depositing && SettlementLib._settleableAssets(payerAssets) == 0)
8         ? 0
9         : assets.mulDiv(settleableR, ownAssets);
10 }

```

As a result, a larger deposit enlarges the payer pool, which reduces the depositor's effective leverage, which in turn shrinks their fee ($fee = assets \times settleableReceiver \div (payerPool + assets)$) and vice versa.

The mint entry point is where this dependency is not handled correctly. mint first derives the fee from the net asset amount and passes the resulting gross amount to _deposit (step 1):

```

1  function previewDeposit(uint256 assets) public view override(IERC4626, ERC4626Upgradeable) returns (uint256) {
2      uint256 fee = Math.min(assets, feeOnTotal(_notional(assets, true), _market.entryFee()));
3      return super.previewDeposit(assets - fee);
4  }
5
6  function deposit(uint256 assets, address receiver) public virtual returns (uint256) {
7      // --SNIP
8      uint256 shares = previewDeposit(assets);
9      _deposit(_msgSender(), receiver, assets, shares);
10     return shares;
11 }

```

_deposit then recomputes the fee from the gross amount (step 2):

```

1  function _deposit(address caller, address receiver, uint256 assets, uint256 shares) internal virtual override {
2      // --SNIP
3      @==> uint256 fee = Math.min(assets, feeOnTotal(_notional(assets, true), _market.entryFee()));
4      IERC20(asset()).safeTransferFrom(caller, address(_market), assets);

```

```

5   _market.recordDeposit(assets, fee, poolKind);
6   _mint(receiver, shares);
7   emit Deposit(caller, receiver, assets, shares);
8 }

```

The fee is thus calculated twice, and the two calculations are expected to agree. In the mint flow, however, they do not, because each step computes the notional from a different amount:

- `previewMint` (step 1) calculates the fee from the smaller, net amount, which corresponds to a higher notional ratio and therefore a higher fee rate. It quotes the user a gross amount to pay based on this fee;
- `_deposit` (step 2) discards that fee and recomputes it from the larger, gross amount, which corresponds to a lower fee rate (because of smaller notional) and therefore a smaller fee;

Because the fee actually charged is smaller than the one quoted when calculating the gross amount, more capital ends up in the pool than the minted shares were priced for. Critically, the share count is fixed in step 1 — before `_deposit` is called — and nothing subsequently adjusts it once the fee is recalculated from the gross amount. The resulting surplus capital sits in the payer pool with no shares attached to it, and is effectively distributed among all existing payer-shareholders. The depositor thereby loses a portion of the assets they paid in, without receiving any shares in return.

Consider the following example, which illustrates the issue: Pre-state: the payer pool holds 5,000 assets, the receiver pool holds 1,000,000 assets, and the entry fee is 0.25%.

- Bob wants a payer position worth 10,000 USDC. The notional for this position is 666,666, and the share count is fixed at this point. Bob calls `mint`, which calls `previewMint` to calculate the fee::
- The fee quoted by `previewMint` is 1,994.015. - The gross amount Bob must pay is 11,994.015.
 - Bob pays 11,994.015 USDC against the shares requested, and this gross amount is passed to `_deposit`;
 - `_deposit` recalculates the fee from this gross amount::
- The notional of 11,994.015 is 705,778.076 - The recalculated fee is: 1,760.045

The fee actually taken (1,760.045) is therefore 233.97 USDC less than the fee originally quoted to Bob. The pool receives 10,233.97 USDC, but the shares minted for Bob remain fixed at the step-1 priced at 10,000 USDC. As a result, Bob effectively loses 233.97 USDC, since no shares back this amount — it is instead attributed to the existing payer-shareholders.

NOTE: Same issue exists in the withdraw flow.

Recommendation(s): Ensure the fee used to determine the minted share count and the fee actually recorded/transferred in `_deposit` are computed from the same basis. Concretely, either:

- Compute the fee once (e.g., in `mint`/`previewMint`) and pass it through to `_deposit` instead of recomputing it there from the gross amount, or;
- Recompute the shares in `_deposit` after the fee is finalised, so that the minted shares always match the fee actually charged;

Status: Fixed

Update from the client: Fixed in commit [9fcf783](#)

6.2 [Low] `_deposit` does not revert on zero shares

File(s): `/src/core/NullV1Pool.sol`

Description: When depositing into an underfunded pool, leverage is high, and the fee charged against the deposited assets can saturate to the full asset amount. As a result, a depositor can be charged a 100% fee on their assets:

```

1  function previewDeposit(uint256 assets) public view override(IERC4626, ERC4626Upgradeable) returns (uint256) {
2      uint256 fee = Math.min(assets, feeOnTotal(_notional(assets, true), _market.entryFee()));
3      return super.previewDeposit(assets - fee);
4  }

```

In this case, the shares minted for the depositor will be zero. The issue is that `_deposit` does not guard against this:

```

1  function _deposit(address caller, address receiver, uint256 assets, uint256 shares) internal virtual override {
2      if (NullV1Factory(_market.factory()).isDepositsPaused()) {
3          revert ErrorsLib.DepositsPaused();
4      }
5      _checkAllowed(caller, true);
6      _checkAllowed(receiver, true);
7      IERC20(asset()).safeTransferFrom(caller, address(_market), assets);
8      uint256 fee = Math.min(assets, feeOnTotal(_notional(assets, true), _market.entryFee()));
9      _market.recordDeposit(assets, fee, poolKind);
10     _mint(receiver, shares);

```

```
11   emit Deposit(caller, receiver, assets, shares);
12 }
```

This allows the depositor to pay a 100% fee while receiving zero shares, resulting in a loss of funds.

Recommendation(s): Add a check in `_deposit` that reverts if shares is zero.

Status: Fixed

Update from the client: Fixed in commit [1f0562a](#)

6.3 [Low] deposit flow is cheaper than mint flow, causing protocol to lose portion of fees

File(s): `/src/core/NullV1Pool.sol`

Description: Both payer and receiver pools charge entry fees against the notional, which is the leverage the deposit is making ($\text{leverage} = \text{settleableReceiver} / (\text{payerPool} + \text{your deposit})$). The higher the deposit is, the less the leverage is:

```
1  function _notional(uint256 assets, bool depositing) private view returns (uint256) {
2      (, uint256 receiverAssets, uint256 payerAssets,,) = _market.state();
3      uint256 ownAssets =
4          (poolKind == NullV1PoolKind.Receiver ? receiverAssets : payerAssets) + (depositing ? assets : 0);
5      if (depositing && poolKind == NullV1PoolKind.Receiver) receiverAssets += assets;
6      uint256 settleableR = SettlementLib._settleableAssets(receiverAssets);
7      return ownAssets == 0 || settleableR == 0 || (depositing && SettlementLib._settleableAssets(payerAssets) == 0)
8          ? 0
9      @==> : assets.mulDiv(settleableR, ownAssets);
10 }
```

When making deposits into the pool, there are two flows: mint which spends the assets based on the requested shares, and deposit, which mints shares based on the requested assets. However, if we take a look at how both functions charge the fees: 1. `previewMint`:

```
1  function previewMint(uint256 shares) public view override(IERC4626, ERC4626Upgradeable) returns (uint256) {
2      uint256 assets = super.previewMint(shares);
3      return assets + feeOnRawNotional(_notional(assets, true), assets, _market.entryFee());
4  }
```

2. `previewDeposit`:

```
1  function previewDeposit(uint256 assets) public view override(IERC4626, ERC4626Upgradeable) returns (uint256) {
2      uint256 fee = Math.min(assets, feeOnTotal(_notional(assets, true), _market.entryFee()));
3      return super.previewDeposit(assets - fee);
4  }
```

`previewMint` calculates the leverage in `_notional` based on the net asset the user will deposit into the pool, while `previewDeposit` **measures the leverage against a gross deposit that includes the fee that is about to be charged**. That fee doesn't go into the pool, but still is included in the leverage calculation, which makes it less and thus gives higher shares. So, for depositing the same amount of assets, the deposit flow will be cheaper than mint flow, in which the former will charge fewer fees than the latter.

Consider the following example demonstrating the issue:

- Pre-state: The receiver pool has 1_000_000 assets and the payer pool has 5_000 assets;
- mint flow:
 - - To deposit a net amount of 10_000, you need spend 11_994,015555 and request shares of 10000000000000000000
 - The fee charged: 11994015555
 - Minted shares: 10000000000000000000
- deposit flow:
 - amount spent: 11994015555
 - The fee charged: 1760045072
 - Minted shares: 102339704830000000000

So, through the deposit flow, the position's value has more 233,97 USDC than the mint flow, **for the same spent amount**.

Recommendation(s): Consider disabling one of the flows. It is tricky to fix the issue while keeping both flows given the notional logic.

Status: Fixed

Update from the client: Fixed in commit [9a57c6a](#)

6.4 [Info] Inconsistent reentrancy guard implementations across contracts

File(s): /src/core/NullV1Factory.sol, /src/core/NullV1Market.sol, /src/periphery/NullV1Router.sol,

Description: Reentrancy protection is implemented across the protocol to prevent untrusted external calls from re-entering state-modifying functions.

However, three distinct versions of reentrancy guards are used across the codebase rather than standardising on the custom NullV1ReentrancyGuard contract.

- NullV1Market uses NullV1ReentrancyGuard;
- NullV1Factory uses ReentrancyGuard which is storage based;
- NullV1Router uses ReentrancyGuardTransient from OZ library;

This inconsistency creates architectural fragmentation and maintenance overhead across the codebase. Additionally, using persistent storage in NullV1Factory incurs higher execution gas costs than transient storage implementations.

Recommendation(s): Consider standardising all contracts across the project to use NullV1ReentrancyGuard as the single transient storage-based reentrancy guard implementation.

Status: Fixed

Update from the client: Fixed in commit [9fcf783](#)

6.5 [Info] No mechanism to prevent deployment of new markets using whitelisted account, if compromised

File(s): /src/core/NullV1Factory.sol

Description: In the NullV1Factory contract, the whitelistAdmin grants market creation privileges to specific addresses by calling the addToWhitelist(...) function. Whitelisted users can then call the createMarket(...) function to instantiate new markets with specific parameters, such as collateral tokens, oracle feeds, and funding rate models.

However, the NullV1Factory contract does not provide a removeFromWhitelist(...) function or any other mechanism to revoke an address's whitelisted status once it has been granted. While deposit functionality can be independently restricted using the setDepositsBlocked(...) function in the NullV1Pool contract via _checkAllowed(...), the createMarket(...) function relies solely on the isWhitelisted mapping without any secondary blacklist control.

```

1  function createMarket(...) external noDelegateCall nonReentrant returns (INullV1Market market) {
2      // ...
3      // @audit-issue The contract checks whitelist status, but there is no mechanism to revoke it.
4      if (!isWhitelisted[msg.sender]) {
5          revert ErrorsLib.NotAllowed(msg.sender);
6      }
7      // ...
8  }
```

This means that if a whitelisted address is compromised or should otherwise be restricted, the whitelistAdmin has no way to revoke its ability to create new markets. The compromised account can continue calling createMarket(...) indefinitely to deploy markets with arbitrary or malicious oracle parameters and funding rate models into the protocol.

Recommendation(s): Consider extending the existing blacklist check to also gate the createMarket(...) function.

Status: Fixed

Update from the client: Fixed in commit [9fcf783](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- **Technical whitepaper:** A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- **User manual:** A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- **Code documentation:** Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- **API documentation:** API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- **Testing documentation:** Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- **Audit documentation:** Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Null Solution's documentation

The Null Solution team provided a comprehensive overview of the main system components during the kick-off call, including a detailed explanation of the intended functionalities and a written specification for the different contracts. Additionally, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and an in-depth understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
$ forge test

Ran 7 tests for test/periphery/utils/NullV1LensCollateralMetrics.t.sol:NullV1LensCollateralMetricsTest
[PASS] test_computeCollateralRate_revertsForNonVault() (gas: 87574)
[PASS] test_computeCollateralRate_skipsAdapterWithWrongParent() (gas: 1197798)
[PASS] test_computeCollateralRate_weightedSupplyApr() (gas: 1208951)
[PASS] test_computeCollateralRate_zeroWhenVaultEmpty() (gas: 1175134)
[PASS] test_computeCollateralSharePrice_normalizesUnderlyingDecimals() (gas: 373626)
[PASS] test_computeCollateralSharePrice_revertsForNonVault() (gas: 88324)
[PASS] test_computeCollateralSharePrice_scalesByConvertToAssets() (gas: 373670)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 7.88ms (2.06ms CPU time)

Ran 2 tests for test/integration/HappyPath.t.sol:HappyPathTest
[PASS] test_baseHappyPath_ShouldWork() (gas: 1240914)
[PASS] test_routerHappyPath_ShouldWorkWithRouter() (gas: 1915142)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 9.11ms (2.53ms CPU time)

Ran 6 tests for test/oracle-latency-arbitrage/03_payer_HighLeverageMustNotProfit.poc.t.sol:PayerLeveragePoC
[PASS] test_payer_Control_BalancedReceiverAttackStillBlockedByFlatFee() (gas: 6598281)
[PASS] test_payer_ExitTrap_SaturatesAtExtremeLeverage() (gas: 269979)
[PASS] test_payer_HighLeverageMustNotBeatMaxFlatFee() (gas: 246882)
[PASS] test_payer_HighLeverageMustNotBeatOnePercentFlatFee() (gas: 257157)
[PASS] test_payer_NearEmptyPoolMustNotProfitOnOnePercentMove() (gas: 1670526)
[PASS] test_payer_NotionalFeeMustCoverLeveragedEdge() (gas: 28201)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 10.63ms (3.74ms CPU time)

Ran 5 tests for test/core/NullV1Market.t.sol:NullV1MarketPriceOracleTest
[PASS] test_constructor_ShouldAllowZeroPrice() (gas: 4728641)
[PASS] test_multipleOperations_ShouldNotBrickWithZeroPrice() (gas: 668095)
[PASS] test_pnlCalculation_ShouldExcludeLargeProtocolFees() (gas: 461674)
[PASS] test_pnlCalculation_ShouldExcludeProtocolFees() (gas: 468894)
[PASS] test_updateState_ShouldHandleZeroToNonZeroPriceTransition() (gas: 458792)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.15ms (2.07ms CPU time)

Ran 19 tests for test/mock/MockMorphoVaultV2.t.sol:MockMorphoVaultV2Test
[PASS] test_accrue_isNoopWithinSameBlock() (gas: 205660)
[PASS] test_deposit_mintsSharesWithDecimalOffset() (gas: 177116)
[PASS] test_indexerFormula_resolvesToConfiguredApr() (gas: 215126)
[PASS] test_integration_usableAsMarketBaseAsset() (gas: 5871576)
[PASS] test_metadata() (gas: 39755)
[PASS] test_redeem_returnsUnderlying() (gas: 178668)
[PASS] test_selfDescribing_marketData() (gas: 212754)
[PASS] test_selfDescribing_topology() (gas: 19495)
[PASS] test_setApr_allowsExactlyMaxRate() (gas: 43208)
[PASS] test_setApr_onlyOwner() (gas: 14268)
[PASS] test_setApr_revertsAboveMaxRate() (gas: 20855)
[PASS] test_setApr_updatesRateAndApr() (gas: 44340)
[PASS] test_setMaxRate_clampsCurrentRate() (gas: 52872)
[PASS] test_setMaxRate_onlyOwner() (gas: 14283)
[PASS] test_setRatePerSecond_onlyOwner() (gas: 14774)
[PASS] test_withdraw_returnsUnderlying() (gas: 209089)
[PASS] test_yield_growsTotalAssetsOverTime() (gas: 204023)
[PASS] test_yield_isProportionalToShares() (gas: 300002)
[PASS] test_yield_isRedeemableAndSolvent() (gas: 207377)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 12.10ms (3.77ms CPU time)

Ran 2 tests for test/mock/MockUSDC.t.sol:MockUSDCTest
[PASS] test_metadata() (gas: 17502)
[PASS] test_mint() (gas: 58434)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 576.58µs (337.67µs CPU time)

Ran 6 tests for test/integration/Lifecycle.t.sol:LifecycleTest
[PASS] test_lifecycle_FundingAccruesAcrossEpochs() (gas: 597208)
[PASS] test_lifecycle_MixedFlowsAndFullUnwind() (gas: 545199)
[PASS] test_lifecycle_NegativeFundingAccruesAcrossEpochs() (gas: 597320)
[PASS] test_lifecycle_NoFeesEverAccrue() (gas: 519112)
[PASS] test_lifecycle_PayerWinsAcrossEpochs() (gas: 577694)
[PASS] test_lifecycle_ReceiverWinsAcrossEpochs() (gas: 600909)
```

Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 13.06ms (5.88ms CPU time)

Ran 1 test for test/fork/MainnetVault.fork.t.sol:MainnetVaultForkTest

[SKIP] test_fork_routerWrapsAndUnwrapsThroughLiveVault() (gas: 0)

Suite result: ok. 0 passed; 0 failed; 1 skipped; finished in 1.11ms (361.92µs CPU time)

Ran 10 tests for test/oracle-latency-arbitrage/01_receiver_LatencyArbMustNotProfit.poc.t.sol:ReceiverUnmitigatedPoC

[PASS] test_receiver_AtomicLatencyArbMustNotProfitAtOnePercentFee() (gas: 249730)

[PASS] test_receiver_AttackAcrossSixtySecondsMustNotProfit() (gas: 255549)

[PASS] test_receiver_AttackOneBlockLaterMustNotProfit() (gas: 255775)

[PASS] test_receiver_FreshPriceAtEntryMustEliminateProfit() (gas: 251536)

[PASS] test_receiver_HoldingPeriodMustNotLeaveProfit() (gas: 312539)

[PASS] test_receiver_HonestOnePercentMoveCostsOnePercentOfReceiverPool() (gas: 146083)

[PASS] test_receiver_OversizedStakeMustNotProfit() (gas: 257823)

[PASS] test_receiver_RealisticEightBpsMustNotProfitAtDefaultFee() (gas: 249612)

[PASS] test_receiver_SmallStakeMustNotProfit() (gas: 257923)

[PASS] test_receiver_StaleDepositThenFreshRedeemMustNotProfit() (gas: 249598)

Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 14.55ms (7.12ms CPU time)

Ran 45 tests for test/core/NullV1Pools.t.sol:NullV1PoolsTest

[PASS] test_deposit_ReceiverBelowFloorPaysNoEntryFee() (gas: 206602)

[PASS] test_deposit_ShouldApplyEntryFee() (gas: 453169)

[PASS] test_deposit_ShouldChargeNoEntryFeeWhenZeroed() (gas: 256222)

[PASS] test_deposit_ShouldCheckAllowlistBeforeBlocklist() (gas: 176998)

[PASS] test_deposit_ShouldCheckPauseBeforeAllowlist() (gas: 135151)

[PASS] test_deposit_ShouldRevertForExistingHolderWhenDepositsBlocked() (gas: 264915)

[PASS] test_deposit_ShouldRevertForExistingHolderWhenDepositsPaused() (gas: 234389)

[PASS] test_deposit_ShouldRevertOnBothPoolsWhenDepositsBlocked() (gas: 204928)

[PASS] test_deposit_ShouldRevertOnBothPoolsWhenDepositsPaused() (gas: 133701)

[PASS] test_deposit_ShouldRevertOnMarketCreatedAfterPause() (gas: 991677)

[PASS] test_deposit_ShouldRevertWhenCallerDepositsBlocked() (gas: 128569)

[PASS] test_deposit_ShouldRevertWhenCallerNotAllowed() (gas: 147826)

[PASS] test_deposit_ShouldRevertWhenDepositsPaused() (gas: 82248)

[PASS] test_deposit_ShouldRevertWhenReceiverDepositsBlocked() (gas: 132156)

[PASS] test_deposit_ShouldRevertWhenReceiverNotAllowed() (gas: 99793)

[PASS] test_deposit_ShouldWorkAgainAfterUnblock() (gas: 292766)

[PASS] test_initialDonationInflationAttack_ShouldPreventInflation() (gas: 444803)

[PASS] test_initialize_ShouldRevertOnZeroMarket() (gas: 2037027)

[PASS] test_initialize_ShouldSetInitialValues() (gas: 59979)

[PASS] test_mint_ShouldApplyEntryFee() (gas: 509219)

[PASS] test_mint_ShouldChargeNoEntryFeeWhenZeroed() (gas: 259038)

[PASS] test_mint_ShouldRevertWhenCallerDepositsBlocked() (gas: 121148)

[PASS] test_mint_ShouldRevertWhenCallerNotAllowed() (gas: 147048)

[PASS] test_mint_ShouldRevertWhenDepositsPaused() (gas: 75415)

[PASS] test_mint_ShouldRevertWhenReceiverDepositsBlocked() (gas: 131753)

[PASS] test_mint_ShouldRevertWhenReceiverNotAllowed() (gas: 99654)

[PASS] test_receiverPool_ShouldQuoteDefaultEntryFee() (gas: 293304)

[PASS] test_recoveryAfterPoolWipeIsSweptByOwnerWhileSharesRemain() (gas: 626777)

[PASS] test_redeem_ShouldChargeNoExitFeeWhenZeroed() (gas: 233135)

[PASS] test_redeem_ShouldRevertWhenOwnerNotAllowed() (gas: 268547)

[PASS] test_redeem_ShouldWorkWhenDepositsBlocked() (gas: 226143)

[PASS] test_redeem_ShouldWorkWhenDepositsPaused() (gas: 205732)

[PASS] test_redeem_ShouldWorkWithExitFee() (gas: 434891)

[PASS] test_reentrancy_ShouldBlockReentrantPoolOperations() (gas: 900168)

[PASS] test_transferFrom_ShouldRevertEvenWithAllowance() (gas: 240542)

[PASS] test_transfer_ShouldRevertEvenBetweenAllowedAddresses() (gas: 215133)

[PASS] test_withdrawAndRedeem_ShouldWork() (gas: 303899)

[PASS] test_withdraw_ShouldChargeNoExitFeeWhenZeroed() (gas: 243321)

[PASS] test_withdraw_ShouldRevertWhenCallerNotAllowed() (gas: 265812)

[PASS] test_withdraw_ShouldRevertWhenReceiverNotAllowed() (gas: 242327)

[PASS] test_withdraw_ShouldWorkForAllowedOwnerViaAllowedSpender() (gas: 292595)

[PASS] test_withdraw_ShouldWorkForBlockedOwnerViaAllowedSpender() (gas: 319988)

[PASS] test_withdraw_ShouldWorkWhenDepositsBlocked() (gas: 260127)

[PASS] test_withdraw_ShouldWorkWhenDepositsPaused() (gas: 235682)

[PASS] test_withdraw_ShouldWorkWithExitFee() (gas: 461783)

Suite result: ok. 45 passed; 0 failed; 0 skipped; finished in 22.37ms (15.39ms CPU time)

Ran 5 tests for test/core/NullV1ReentrancyGuard.t.sol:NullV1ReentrancyGuardTest

[PASS] test_guarded_ShouldAllowNonReentrantCall() (gas: 33983)

[PASS] test_guarded_ShouldBlockDirectReentrancy() (gas: 32851)

[PASS] test_guarded_ShouldBlockReentrancyThroughSplitHelpers() (gas: 32460)

[PASS] test_guarded_ShouldReleaseGuardAfterRevertingCall() (gas: 58128)

[PASS] test_isEntered_ShouldReflectGuardStateDuringCall() (gas: 58970)

Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 202.12µs (93.21µs CPU time)

```
Ran 9 tests for test/core/NotionalFeeEdges.t.sol:NotionalFeeEdgesTest
[PASS] test_firstPayerDeposit_IsFreeThenPastSingularity() (gas: 403488)
[PASS] test_higherExitFee_SaturatesAtLowerLeverage() (gas: 473440)
[PASS] test_lock_IsRecoverableOnlyByAFullSizedRescueDeposit() (gas: 574225)
[PASS] test_losingPayerLps_RedeemForZeroWhenPastSingularity() (gas: 458127)
[PASS] test_payerEntry_IsFreeBeforeReceiverPoolIsFunded() (gas: 366225)
[PASS] test_previewMint_QuotesDepositingNotionalFeeWhenSeeded() (gas: 457626)
[PASS] test_previewMint_RoundTripsWithDepositWhenSeeded() (gas: 359182)
[PASS] test_receiverEntry_IsFreeAtOrBelowFloor() (gas: 418949)
[PASS] test_receiverEntry_IsFreeWhilePayerPoolUnfunded() (gas: 220098)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 9.97ms (9.05ms CPU time)

Ran 5 tests for test/periphery/oracle/NullV1ChainlinkOracleFactory.t.sol:NullV1ChainlinkOracleV11FactoryTest
[PASS] test_constructor_ShouldRevertOnZeroVerifierProxy() (gas: 37848)
[PASS] test_createOracle_ShouldCreateOracle() (gas: 286338)
[PASS] test_createOracle_ShouldSetCorrectFeedIds() (gas: 278800)
[PASS] test_createOracle_ShouldSetCorrectPriceDecimals() (gas: 274535)
[PASS] test_createOracle_ShouldSetCorrectVerifierProxy() (gas: 276676)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 459.33µs (206.88µs CPU time)

Ran 45 tests for test/periphery/oracle/NullV1ChainlinkOracleV11.t.sol:NullV1ChainlinkOracleV11Test
[PASS] test_constructor_revertsOnZeroVerifierProxy() (gas: 37146)
[PASS] test_displayName_matchesUi() (gas: 11793)
[PASS] test_getUpdateFee_returnsZero() (gas: 13843)
[PASS] test_initialize_revertsWhenExtendedEqualsOvernight() (gas: 82492)
[PASS] test_initialize_revertsWhenExtendedFeedIdIsZero() (gas: 82444)
[PASS] test_initialize_revertsWhenOvernightFeedIdIsZero() (gas: 81716)
[PASS] test_initialize_revertsWhenPriceDecimalsAboveScale() (gas: 82560)
[PASS] test_initialize_revertsWhenRegularEqualsExtended() (gas: 83024)
[PASS] test_initialize_revertsWhenRegularEqualsOvernight() (gas: 82258)
[PASS] test_initialize_revertsWhenRegularFeedIdIsZero() (gas: 81522)
[PASS] test_price_normalizesTo1e36() (gas: 439959)
[PASS] test_price_returnsLastPriceAfterEscapePeriod() (gas: 442785)
[PASS] test_price_revertsWhenNoReportStored() (gas: 14552)
[PASS] test_price_revertsWhenReportTimestampFarInFuture() (gas: 442674)
[PASS] test_price_revertsWhenStale() (gas: 442971)
[PASS] test_price_revertsWhenStaleJustBeforeEscapePeriod() (gas: 442341)
[PASS] test_price_routesOvernightToOvernightFeed() (gas: 444530)
[PASS] test_price_routesPostMarketToExtendedFeed() (gas: 442654)
[PASS] test_price_routesPreMarketToExtendedFeed() (gas: 442279)
[PASS] test_price_toleratesSlightlyFutureReportTimestamp() (gas: 442927)
[PASS] test_update_acceptedOneSecondBeforeEscapeHatchOpens() (gas: 467489)
[PASS] test_update_acceptsWhenReportTimestampInFuture() (gas: 436831)
[PASS] test_update_bufferedOutgoingFeedReportsCannotWalkPriceBackwards() (gas: 548269)
[PASS] test_update_crossFeedSameTimestampDoesNotOverwrite() (gas: 495338)
[PASS] test_update_firstReportAcceptedAtLiveChainClock() (gas: 444207)
[PASS] test_update_ignoredAcrossWholeIntakeWindowAfterHatchOpens() (gas: 1242098)
[PASS] test_update_ignoredForeverOnceEscapeHatchOpen() (gas: 574351)
[PASS] test_update_ignoresReplayedOlderReportOnTheSameFeed() (gas: 462757)
[PASS] test_update_revertsInvalidReportVersion() (gas: 284307)
[PASS] test_update_revertsWhenReportTimestampTooFarInFuture() (gas: 391512)
[PASS] test_update_revertsWhenUpdateDataEmpty() (gas: 12895)
[PASS] test_update_revertsWrongFeedId() (gas: 395552)
[PASS] test_update_sameTimestampReportIsNoOp() (gas: 461520)
[PASS] test_update_skipsReportWhenFeedIdDoesNotMatchMarketStatus() (gas: 406607)
[PASS] test_update_skipsWhenMarketClosed() (gas: 396371)
[PASS] test_update_skipsWhenMarketUnknown() (gas: 376537)
[PASS] test_update_skipsWhenMidIsNegative() (gas: 371089)
[PASS] test_update_skipsWhenMidIsZero() (gas: 351074)
[PASS] test_update_skipsWhenMidOverflowsNormalization() (gas: 618992)
[PASS] test_update_skipsWhenObservationsTimestampTooOld() (gas: 392007)
[PASS] test_update_stackedInSameBlockDoesNotRevertEachOther() (gas: 470228)
[PASS] test_update_stackedSameTimestampDoesNotOverwrite() (gas: 461265)
[PASS] test_update_storesMidPrice() (gas: 442990)
[PASS] test_update_switchesActiveFeedOnSessionChange() (gas: 495291)
[PASS] test_update_trailingNewlyActiveFeedDoesNotRegressServedPrice() (gas: 525213)
Suite result: ok. 45 passed; 0 failed; 0 skipped; finished in 12.79ms (12.40ms CPU time)

Ran 36 tests for test/core/NullV1Factory.t.sol:NullV1FactoryTest
[PASS] test_addToWhitelist_ShouldAllowAddressAndEmit() (gas: 42325)
[PASS] test_addToWhitelist_ShouldRevertIfNotWhitelistAdmin() (gas: 13930)
[PASS] test_addToWhitelist_ShouldRevertIfZeroAddress() (gas: 14287)
[PASS] test_checkContractIsMarket_ShouldReturnTrueIfContractIsMarket() (gas: 647376)
[PASS] test_createMarket_ShouldAllowDistinctFundingRateModelsForSamePair() (gas: 1268396)
[PASS] test_createMarket_ShouldAllowMaxSensitivityModel() (gas: 910522)
```

```
[PASS] test_createMarket_ShouldAllowWhitelistedNonOwner() (gas: 675999)
[PASS] test_createMarket_ShouldCheckWhitelistBeforeValidatingArguments() (gas: 21998)
[PASS] test_createMarket_ShouldCreateMarket() (gas: 651837)
[PASS] test_createMarket_ShouldInitializeFundingRateModel() (gas: 771149)
[PASS] test_createMarket_ShouldRevertForOwnerWhenNotWhitelisted() (gas: 4138300)
[PASS] test_createMarket_ShouldRevertIfCallerNotWhitelisted() (gas: 27529)
[PASS] test_createMarket_ShouldRevertIfCollateralIsZeroAddress() (gas: 23858)
[PASS] test_createMarket_ShouldRevertIfCuratorIsZeroAddress() (gas: 23525)
[PASS] test_createMarket_ShouldRevertIfFrmIsNotEnabled() (gas: 267316)
[PASS] test_createMarket_ShouldRevertIfFrmIsZero() (gas: 23805)
[PASS] test_createMarket_ShouldRevertIfMarketAlreadyExists() (gas: 652815)
[PASS] test_enableFrm_ShouldRevertIfAlreadyEnabled() (gas: 18987)
[PASS] test_enableFrm_ShouldRevertIfNotOwner() (gas: 252467)
[PASS] test_enableFrm_ShouldRevertIfZeroAddress() (gas: 14750)
[PASS] test_isDepositsBlocked_ShouldDefaultToFalse() (gas: 19837)
[PASS] test_isDepositsPaused_ShouldDefaultToFalse() (gas: 11784)
[PASS] test_setDepositsBlocked_ShouldBlockAddressAndEmit() (gas: 68191)
[PASS] test_setDepositsBlocked_ShouldFollowWhitelistAdminRotation() (gas: 50110)
[PASS] test_setDepositsBlocked_ShouldNotBlockMarketCreation() (gas: 673242)
[PASS] test_setDepositsBlocked_ShouldRevertIfNotWhitelistAdmin() (gas: 14521)
[PASS] test_setDepositsBlocked_ShouldUnblockAddressAndEmit() (gas: 32440)
[PASS] test_setPauseDeposits_ShouldBeIdempotent() (gas: 24474)
[PASS] test_setPauseDeposits_ShouldNotBlockMarketCreation() (gas: 655359)
[PASS] test_setPauseDeposits_ShouldPauseDepositsAndEmit() (gas: 21700)
[PASS] test_setPauseDeposits_ShouldRevertIfNotOwner() (gas: 17331)
[PASS] test_setPauseDeposits_ShouldRevertIfWhitelistAdmin() (gas: 23612)
[PASS] test_setWhitelistAdmin_ShouldRevertIfNotOwner() (gas: 13617)
[PASS] test_setWhitelistAdmin_ShouldRevertIfZeroAddress() (gas: 13826)
[PASS] test_setWhitelistAdmin_ShouldRotateAdminAndEmit() (gas: 52490)
[PASS] test_whitelistAdmin_ShouldDefaultToDeployer() (gas: 14087)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 3.66ms (2.49ms CPU time)
```

Ran 10 tests for test/periphery/frm/NullV1ImbalanceFundingRateModel.t.sol:NullV1ImbalanceFundingRateModelTest

```
[PASS] test_constructor_ShouldRevertOnMuAboveWad() (gas: 36501)
[PASS] test_constructor_ShouldRevertOnTargetCoverageAboveWad() (gas: 36367)
[PASS] test_constructor_ShouldRevertOnZeroMu() (gas: 36169)
[PASS] test_constructor_ShouldRevertOnZeroTargetCoverage() (gas: 36498)
[PASS] test_constructor_ShouldStoreParameters() (gas: 245848)
[PASS] test_fundingRateView_ShouldMatchFundingRate() (gas: 251987)
[PASS] test_fundingRate_ShouldFollowSignedImbalanceFormula() (gas: 252519)
[PASS] test_fundingRate_ShouldReturnZeroAtTargetCoverage() (gas: 249841)
[PASS] test_fundingRate_ShouldReturnZeroWhenEitherPoolIsEmpty() (gas: 248827)
[PASS] test_fundingRate_ShouldScaleWithMu() (gas: 249742)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 328.92µs (227.79µs CPU time)
```

Ran 15 tests for test/periphery/utills/NullV1Lens.t.sol:NullV1LensTest

```
[PASS] test_computeAdapterWeightedApr_ShouldReturnZeroAprWhenMarketHasNoBorrow() (gas: 701731)
[PASS] test_computeAdapterWeightedApr_ShouldReturnZeroAprWhenMarketHasNoSupply() (gas: 681828)
[PASS] test_computeAdapterWeightedApr_ShouldReturnZeroForForeignAdapter() (gas: 553540)
[PASS] test_computeAdapterWeightedApr_ShouldSkipMarketsWithZeroExpectedSupply() (gas: 749031)
[PASS] test_previewAll_ShouldPreviewAll() (gas: 423276)
[PASS] test_previewAll_ShouldZeroOutFailedPreviews() (gas: 20897)
[PASS] test_previewMarket_ShouldWork() (gas: 413682)
[PASS] test_simulateAll_ShouldRevertOnLengthMismatch() (gas: 16412)
[PASS] test_simulateAll_ShouldSimulateAll() (gas: 422112)
[PASS] test_simulateAll_ShouldZeroOutFailedSimulations() (gas: 21548)
[PASS] test_simulateMarket_ShouldRevertWithZeroPrice() (gas: 11652)
[PASS] test_simulateMarket_ShouldWork() (gas: 413825)
[PASS] test_simulateState_ShouldMatchComputeUpdateWithCuratorPrice() (gas: 64098)
[PASS] test_simulateState_ShouldRevertWithZeroPrice() (gas: 37528)
[PASS] test_simulateState_ShouldSimulateUpdateWithExternalPrice() (gas: 59111)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 3.90ms (2.97ms CPU time)
```

Ran 2 tests for test/core/NullV1NoDelegateCall.t.sol:NullV1NoDelegateCallTest

```
[PASS] test_createMarket_ShouldRevertViaDelegateCall() (gas: 25166)
[PASS] test_createMarket_ShouldWorkViaDirectCall() (gas: 645183)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 702.08µs (152.08µs CPU time)
```

Ran 17 tests for test/oracle-latency-arbitrage/02_receiverFee_CounterpartyFlatFee.poc.t.sol:ReceiverFeePoC

```
[PASS] test_receiverFee_BreakevenIndependentOfPoolImbalance() (gas: 330548)
[PASS] test_receiverFee_CannotRecycleFeeByOwningBothSides() (gas: 6997984)
[PASS] test_receiverFee_DepositCreditsOppositePoolNotProtocol() (gas: 186000)
[PASS] test_receiverFee_EmptyCounterpartySettlementStaysReconciled() (gas: 1249501)
[PASS] test_receiverFee_EmptyCounterpartyWaivesFeeIntoOwnPool() (gas: 1170373)
[PASS] test_receiverFee_EntryOnlyFeeLeavesExitTimingAttackOpen() (gas: 304179)
```

```
[PASS] test_receiverFee_EqualSumSplitsShareAlmostTheSameBand() (gas: 19768668)
[PASS] test_receiverFee_ExitFeeDoesNotStopExitTimingAttack() (gas: 6845709)
[PASS] test_receiverFee_HonestRoundTripCostsAboutTwoPercentAtCap() (gas: 239004)
[PASS] test_receiverFee_MinimalSafeFeeIsHalfTheMovePerSide() (gas: 197115669)
[PASS] test_receiverFee_OneBpBelowBreakevenStillProfitable() (gas: 13191104)
[PASS] test_receiverFee_OnePercentFeeBlocksOnePercentAttack() (gas: 251272)
[PASS] test_receiverFee_OnePercentFeeBlocksRealisticEightBpsAttack() (gas: 250928)
[PASS] test_receiverFee_OversizedAttackLosesMoreThanEntirePayerPool() (gas: 258101)
[PASS] test_receiverFee_PassiveCounterpartyLpEarnsFromChurn() (gas: 248223)
[PASS] test_receiverFee_TwentyFiveBpsBlocksRoutineButNotOnePercentTail() (gas: 6852819)
[PASS] test_receiverFee_WithdrawCreditsOppositePoolNotProtocol() (gas: 171486)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 66.50ms (65.42ms CPU time)

Ran 6 tests for test/periphery/Utils/NullV1Multicall.t.sol:NullV1MulticallTest
[PASS] test_previewAll_ShouldPreviewMultipleMarkets() (gas: 198212)
[PASS] test_previewAll_ShouldReturnEmptyArrayForEmptyBatch() (gas: 12607)
[PASS] test_previewAll_ShouldZeroOutMarketWithRevertingOracle() (gas: 117233)
[PASS] test_previewAll_ShouldZeroOutOnlyFailedEntriesInBatch() (gas: 181826)
[PASS] test_simulateAll_ShouldSimulateMultipleMarketsWithDistinctPrices() (gas: 166432)
[PASS] test_simulateAll_ShouldZeroOutOnlyFailedEntriesInBatch() (gas: 130423)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 3.24ms (994.92µs CPU time)

Ran 8 tests for test/core/ProtocolFeeRouting.t.sol:ProtocolFeeRoutingEnabledTest
[PASS] test_accounting_ReconcilesAfterChurn() (gas: 314300)
[PASS] test_allowProtocolFees_IsObservedByEveryClone() (gas: 967040)
[PASS] test_allowProtocolFees_MatchesTheFactoryConfiguration() (gas: 11831)
[PASS] test_emptyCounterparty_ChargesNoReceiverFee() (gas: 1164981)
[PASS] test_entryFee_AccruesToProtocolFees() (gas: 184916)
[PASS] test_exitFee_AccruesToProtocolFees() (gas: 192779)
[PASS] test_fuzz_entryFee_AlwaysAccruesToTheProtocol(uint256,uint16) (runs: 256, : 213878, ~: 216013)
[PASS] test_sweepFees_TransfersTheAccruedEntryAndExitFees() (gas: 262824)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 141.54ms (140.62ms CPU time)

Ran 1 test for test/fork/SepoliaChainlink.fork.t.sol:SepoliaChainlinkForkTest
[SKIP] test_fork_updateAndPrice_withPinnedSignedReports() (gas: 0)
Suite result: ok. 0 passed; 0 failed; 1 skipped; finished in 197.25µs (25.58µs CPU time)

Ran 2 tests for test/core/NullV1PoolNaming.t.sol:NullV1PoolNamingTest
[PASS] test_poolNaming_UsesFallbackNamesWithoutOracleMetadata() (gas: 1110092)
[PASS] test_poolNaming_WithEmptyBaseMetadata() (gas: 1070631)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 881.21µs (353.67µs CPU time)

Ran 60 tests for test/core/SettlementLib.t.sol:SettlementLibTest
[PASS] test_constants() (gas: 12355)
[PASS] test_finalizeUpdate_Discontinuity_BreakerFiresAndPriceAdvances() (gas: 59126)
[PASS] test_finalizeUpdate_PnLAfterDeposit() (gas: 394742)
[PASS] test_finalizeUpdate_SameBlock_DiscontinuitySkipsPnL() (gas: 59697)
[PASS] test_finalizeUpdate_SameBlock_SecondSettlementIsIncremental() (gas: 62352)
[PASS] test_finalizeUpdate_SameBlock_SettlesPnLInFull() (gas: 59930)
[PASS] test_finalizeUpdate_ShouldApplyFundingAndReturns() (gas: 393534)
[PASS] test_finalizeUpdate_ShouldFollowExpectedFundingFormula() (gas: 393762)
[PASS] test_finalizeUpdate_ShouldHandleDifferentDecimals() (gas: 802366)
[PASS] test_finalizeUpdate_ShouldHandleLargePriceMove() (gas: 395216)
[PASS] test_finalizeUpdate_ShouldNetOppositeSignFundingAndPnL() (gas: 67957)
[PASS] test_finalizeUpdate_ShouldReinforceSameSignFundingAndPnL() (gas: 66492)
[PASS] test_finalizeUpdate_ShouldRevertOnZeroPrice() (gas: 56693)
[PASS] test_finalizeUpdate_ShouldSkipFundingAndPnLWhenPayerPoolIsZero() (gas: 368932)
[PASS] test_finalizeUpdate_ShouldSkipFundingAndPnLWhenReceiverPoolIsZero() (gas: 368830)
[PASS] test_fuzz_finalizeUpdate(uint256,uint256,uint256,uint256,uint32) (runs: 256, : 401521, ~: 402223)
[PASS] test_fuzz_finalizeUpdate_Funding(uint128,uint128,uint32) (runs: 256, : 76897, ~: 76995)
[PASS] test_fuzz_updatePoolsForBalanceChangesLoss_AlwaysConverges(uint128,uint128,uint128) (runs: 256, : 61243, ~: 62977)
[PASS] test_fuzz_updatePoolsForCollateralFee_NeverBreachesFloor(uint128,uint128,uint16,uint32) (runs: 256, : 86551, ~: 88042)
[PASS] test_imbalanceFundingRateModel_ShouldBeSignedByImbalance() (gas: 16023)
[PASS] test_updatePoolsForBalanceChangesGain_ShouldAddProportionallyToPools() (gas: 57010)
[PASS] test_updatePoolsForBalanceChangesLoss_EqualPools_LeftoverToReceiverByTieBreaker() (gas: 57774)
[PASS] test_updatePoolsForBalanceChangesLoss_LeftoverDeductedFromLargestPool_PayerIsLargest() (gas: 57503)
[PASS] test_updatePoolsForBalanceChangesLoss_LeftoverDeductedFromLargestPool_ReceiverIsLargest() (gas: 57429)
[PASS] test_updatePoolsForBalanceChangesLoss_LeftoverOf2Wei_DistributedAcrossPools() (gas: 57628)
[PASS] test_updatePoolsForBalanceChangesLoss_LeftoverRequiresMultipleIterations() (gas: 56714)
[PASS] test_updatePoolsForBalanceChangesLoss_LeftoverWithOnePoolZero() (gas: 38141)
[PASS] test_updatePoolsForBalanceChangesLoss_ShouldAbsorbAllTheLoss() (gas: 56421)
[PASS] test_updatePoolsForBalanceChangesSmallerProtocolPoolZero_ShouldSubProportionallyFromReceiverAndPayerPools()
→ (gas: 56917)
```

```
[PASS] test_updatePoolsForBalanceChanges_RecoveryAfterZeroingIsDivertedToFees() (gas: 60688)
[PASS] test_updatePoolsForBalanceChanges_ShouldCapProtocolFeesAtBalance() (gas: 78509)
[PASS] test_updatePoolsForBalanceChanges_ShouldCreditSurplusToProtocolFeesWhenPoolsEmpty() (gas: 37455)
[PASS] test_updatePoolsForBalanceChanges_ShouldHandleExternalGain() (gas: 57201)
[PASS] test_updatePoolsForBalanceChanges_ShouldIgnoreLossWhenPoolsAreZero() (gas: 17444)
[PASS] test_updatePoolsForBalanceChanges_ShouldKeepPoolsWhenTotalAssetsIsEqual() (gas: 56251)
[PASS] test_updatePoolsForBalanceChanges_ShouldSubProportionallyFromReceiverAndPayerPools() (gas: 57342)
[PASS] test_updatePoolsForCollateralFee_ShouldAccrueFullFeeBelowCap() (gas: 59376)
[PASS] test_updatePoolsForCollateralFee_ShouldNotDebitPoolAtOrBelowFloor() (gas: 60347)
[PASS] test_updatePoolsForCollateralFee_ShouldStopAtFloorWhenFeeExceedsSettleable() (gas: 59200)
[PASS] test_updatePoolsForFundingAndReturns_ShouldConserveTotalAssetsWhenCapped() (gas: 67039)
[PASS] test_updatePoolsForFunding_ShouldCapFeeAtPayerBalanceWhenNegative() (gas: 304558)
[PASS] test_updatePoolsForFunding_ShouldCapFeeAtReceiverBalance() (gas: 65525)
[PASS] test_updatePoolsForFunding_ShouldNotDebitPayerAlreadyBelowFloor() (gas: 65814)
[PASS] test_updatePoolsForFunding_ShouldNotDebitReceiverAlreadyBelowFloor() (gas: 64503)
[PASS] test_updatePoolsForFunding_ShouldSkipWhenPayerPoolIsZero() (gas: 44713)
[PASS] test_updatePoolsForFunding_ShouldSkipWhenReceiverPoolIsZero() (gas: 45384)
[PASS] test_updatePoolsForFunding_ShouldTransferFromPayerToReceiverWhenPayerRich() (gas: 71152)
[PASS] test_updatePoolsForFunding_ShouldTransferFromReceiverToPayer() (gas: 70044)
[PASS] test_updatePoolsForReturns_CeilingBoundary() (gas: 64782)
[PASS] test_updatePoolsForReturns_Discontinuity_SkipsPnLAndJumpsReference() (gas: 62863)
[PASS] test_updatePoolsForReturns_SettlementIsElapsedIndependent() (gas: 59509)
[PASS] test_updatePoolsForReturns_ShouldCapTransferToPayerPool() (gas: 57450)
[PASS] test_updatePoolsForReturns_ShouldDebitReceiverIntoPayerWhenPriceDown() (gas: 58256)
[PASS] test_updatePoolsForReturns_ShouldNotDebitPoolAlreadyAtFloor() (gas: 57452)
[PASS] test_updatePoolsForReturns_ShouldPayReceiverFromPayerWhenPriceUp() (gas: 58786)
[PASS] test_updatePoolsForReturns_ShouldSettleInFullBelowCeiling() (gas: 60315)
[PASS] test_updatePoolsForReturns_ShouldSkipWhenPayerPoolIsZero() (gas: 39395)
[PASS] test_updatePoolsForReturns_ShouldSkipWhenPriceUnchanged() (gas: 55924)
[PASS] test_updatePoolsForReturns_ShouldWorkForExtremePrices() (gas: 56599)
[PASS] test_updatePoolsForReturns_SliverPoolEntitlementMatchesLiability() (gas: 62365)
Suite result: ok. 60 passed; 0 failed; 0 skipped; finished in 134.50ms (134.10ms CPU time)

Ran 27 tests for test/periphery/utils/NullV1Router.t.sol:NullV1RouterTest
[PASS] testFuzz_residualBalance_isZeroForAnyAmounts(uint256,uint256) (runs: 256, : 655766, ~: 656345)
[PASS] test_depositMarket_ShouldDepositToBothPools() (gas: 378679)
[PASS] test_depositMarket_ShouldHandleOnlyPayerDeposit() (gas: 248011)
[PASS] test_depositMarket_ShouldHandleOnlyReceiverDeposit() (gas: 245626)
[PASS] test_depositMarket_ShouldHandleZeroAmounts() (gas: 28233)
[PASS] test_depositMarket_ShouldPassWithValidSlippageTolerance() (gas: 564262)
[PASS] test_depositMarket_ShouldProtectAgainstSlippageAfterPriceChange() (gas: 542497)
[PASS] test_depositMarket_ShouldRevertWhenDeadlinePassed() (gas: 15456)
[PASS] test_depositMarket_ShouldRevertWhenDepositsPaused() (gas: 144177)
[PASS] test_depositMarket_ShouldRevertWhenOracleDemandsUpdateFee() (gas: 5300375)
[PASS] test_depositMarket_ShouldRevertWhenPayerSlippageExceeded() (gas: 563271)
[PASS] test_depositMarket_ShouldRevertWhenReceiverSlippageExceeded() (gas: 429827)
[PASS] test_depositMarket_ShouldRevertWithStaleSlippageAfterPriceChange() (gas: 405619)
[PASS] test_depositMarket_ShouldUpdateOracle() (gas: 369653)
[PASS] test_depositMarket_ShouldWrapUnderlyingBeforeDepositingToPools() (gas: 6880057)
[PASS] test_redeemMarket_ShouldHandleOnlyPayerRedeem() (gas: 426611)
[PASS] test_redeemMarket_ShouldHandleOnlyReceiverRedeem() (gas: 425650)
[PASS] test_redeemMarket_ShouldPassWithValidSlippageTolerance() (gas: 499398)
[PASS] test_redeemMarket_ShouldRedeemSharesFromBothPools() (gas: 493974)
[PASS] test_redeemMarket_ShouldRevertIfInsufficientShares() (gas: 388608)
[PASS] test_redeemMarket_ShouldRevertWhenDeadlinePassed() (gas: 235333)
[PASS] test_redeemMarket_ShouldRevertWhenPayerSlippageExceeded() (gas: 605793)
[PASS] test_redeemMarket_ShouldRevertWhenReceiverSlippageExceeded() (gas: 443811)
[PASS] test_redeemMarket_ShouldUnwrapMarketAssetsBeforePayingUser() (gas: 6933849)
[PASS] test_redeemMarket_ShouldWorkWhenDepositsPaused() (gas: 472549)
[PASS] test_residualBalance_isZeroAfterDepositAndRedeem() (gas: 488017)
[PASS] test_residualBalance_isZeroAfterWrappedDepositAndRedeem() (gas: 6909229)
Suite result: ok. 27 passed; 0 failed; 0 skipped; finished in 565.83ms (564.94ms CPU time)

Ran 11 tests for test/core/ProtocolFeeRouting.t.sol:ProtocolFeeRoutingDisabledTest
[PASS] test_accounting_ReconcilesAfterChurn() (gas: 298133)
[PASS] test_allowProtocolFees_IsObservedByEveryClone() (gas: 967103)
[PASS] test_allowProtocolFees_MatchesTheFactoryConfiguration() (gas: 11894)
[PASS] test_donationToAnEmptyMarketStillAccruesToProtocolFees() (gas: 1027001)
[PASS] test_emptyCounterparty_ChargesNoReceiverFee() (gas: 1165048)
[PASS] test_entryFee_CreditsTheCounterpartyPool() (gas: 168362)
[PASS] test_exitFee_CreditsTheCounterpartyPool() (gas: 175812)
[PASS] test_flooredCounterparty_ChargesNoReceiverFee() (gas: 1314873)
[PASS] test_fuzz_counterpartyAtOrBelowFloor_ChargesNoReceiverFee(uint256) (runs: 256, : 1316383, ~: 1316543)
[PASS] test_fuzz_entryFee_AlwaysLandsInTheCounterpartyPool(uint256,uint16) (runs: 256, : 198873, ~: 199172)
[PASS] test_sweepFees_HasNothingToSweepAfterChurn() (gas: 660418)
```

Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 442.83ms (441.78ms CPU time)

```
Ran 44 tests for test/core/NullV1Market.t.sol:NullV1MarketTest
[PASS] test_UpdateState_ShouldEmit() (gas: 286240)
[PASS] test_depositWithFrom_ShouldDepositFromSpecifiedAddress() (gas: 350645)
[PASS] test_deposit_ShouldHandleOnlyPayerDeposit() (gas: 214123)
[PASS] test_deposit_ShouldHandleOnlyReceiverDeposit() (gas: 210917)
[PASS] test_deposit_ShouldIncrementPools() (gas: 351673)
[PASS] test_deposit_ShouldMintPoolShares() (gas: 360768)
[PASS] test_fees_ShouldDefaultToTheProtocolDefaults() (gas: 17426)
[PASS] test_fees_ShouldRejectAnyCollateralFee() (gas: 72228)
[PASS] test_fees_ShouldUpdateFees() (gas: 97855)
[PASS] test_fuzz_deposit(uint256,uint256) (runs: 256, : 372545, ~: 380040)
[PASS] test_fuzz_entryFeeRateOnlyMovesTheSplit(uint256,uint256,uint16) (runs: 256, : 405151, ~: 405145)
[PASS] test_fuzz_entryFeesStayInsideTheMarket(uint256,uint256) (runs: 256, : 360243, ~: 360284)
[PASS] test_fuzz_roundTripKeepsEveryFeeInTheMarket(uint256,uint256) (runs: 256, : 468051, ~: 468127)
[PASS] test_fuzz_setFees_ShouldRejectAnyFeeAboveTheCaps(uint16,uint16,uint16) (runs: 256, : 77012, ~: 76982)
[PASS] test_fuzz_withdraw(uint256,uint256) (runs: 256, : 480148, ~: 485142)
[PASS] test_inflationAttack_ShouldPreventInflation() (gas: 459888)
[PASS] test_initialize_ShouldRevertOnZeroCollateral() (gas: 3504339)
[PASS] test_initialize_ShouldRevertOnZeroOracle() (gas: 3504072)
[PASS] test_initialize_ShouldSetInitialValues() (gas: 51173)
[PASS] test_mint_ShouldIncrementPools() (gas: 568187)
[PASS] test_onlyPools_ShouldRevertWhenCallerIsNotPool() (gas: 67716)
[PASS] test_previewState_ShouldReturnCorrectValues() (gas: 411502)
[PASS] test_redeem_ShouldDecrementPools() (gas: 463310)
[PASS] test_setFees_ShouldAcceptTheMaximumAndEmit() (gas: 76751)
[PASS] test_setFees_ShouldRevertWhenCollateralFeeAboveMax() (gas: 66903)
[PASS] test_setFees_ShouldRevertWhenEntryFeeAboveMax() (gas: 67007)
[PASS] test_setFees_ShouldRevertWhenExitFeeAboveMax() (gas: 66358)
[PASS] test_setFees_ShouldRevertWhenNotProtocolOwner() (gas: 24817)
[PASS] test_sweepFees_ShouldRevertWhenNotProtocolOwner() (gas: 353926)
[PASS] test_totalAssets_ShouldReflectAccrual() (gas: 348812)
[PASS] test_updatePools_ShouldHandleLosses() (gas: 498167)
[PASS] test_updateState_ShouldAccrueNoCollateralFee() (gas: 407758)
[PASS] test_updateState_ShouldApplyFundingOverTime() (gas: 433100)
[PASS] test_updateState_ShouldApplyNegativeFundingOverTime() (gas: 432059)
[PASS] test_updateState_ShouldApplyNoFundingAtTargetCoverage() (gas: 429769)
[PASS] test_updateState_ShouldRevertWhenMarketNotRegisteredInFactory() (gas: 4112236)
[PASS] test_updateState_ShouldRevertWhenOraclePriceDropsToZero() (gas: 417961)
[PASS] test_updateState_ShouldSkipFundingWhenDepositsPaused() (gas: 426754)
[PASS] test_updateState_ShouldSkipPricePnLWhenDepositsPaused() (gas: 431803)
[PASS] test_withdrawWithTo_ShouldSendToSpecifiedAddress() (gas: 465721)
[PASS] test_withdraw_ShouldDecrementPools() (gas: 483929)
[PASS] test_withdraw_ShouldHandleOnlyPayerWithdraw() (gas: 416798)
[PASS] test_withdraw_ShouldHandleOnlyReceiverWithdraw() (gas: 416143)
[PASS] test_withdraw_ShouldWorkWithZeroOracleWhenDepositsPaused() (gas: 231182)
Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in 1.26s (1.29s CPU time)
```

```
Ran 4 tests for test/core/NullV1Pool.edge.t.sol:NullV1PoolEdgeTest
[PASS] test_afterStaleDeposit_WithdrawShouldWork() (gas: 1826671)
[PASS] test_inflationAttack_ShouldExperiment() (gas: 1058124472)
[PASS] test_realWorldScenario_ShouldWork() (gas: 1576865)
[PASS] test_sharePriceOnEmptyPool_ShouldBeDust() (gas: 660241)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.77s (3.78s CPU time)
```

```
Ran 2 tests for test/oracle-latency-arbitrage/04_fuzz_LatencyArbMustNotProfit.t.sol:LatencyArbFuzzPoC
[PASS] testFuzz_payer_LatencyArbMustNotProfitInsideFeeBand(uint256,uint256,uint256,uint16,uint16,uint16) (runs: 256, :
  ↳ 1491209, ~: 1491652)
[PASS] testFuzz_receiver_LatencyArbMustNotProfitInsideFeeBand(uint256,uint256,uint256,uint16,uint16,uint16) (runs: 256,
  ↳ : 1490369, ~: 1490410)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 47.31s (47.62s CPU time)
```

```
Ran 26 tests for test/core/NullV1PoolERC4626Prop.t.sol:NullV1PoolERC4626PropTest
[PASS] test_RT_deposit_redeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 887061, ~: 890209)
[PASS] test_RT_deposit_withdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 895064, ~: 897560)
[PASS] test_RT_mint_redeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 891015, ~: 893562)
[PASS] test_RT_mint_withdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 897249, ~: 898994)
[PASS] test_RT_redeem_deposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 886881, ~: 888843)
[PASS] test_RT_redeem_mint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 886885, ~: 889092)
[PASS] test_RT_withdraw_deposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 899516, ~: 902132)
[PASS] test_RT_withdraw_mint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 898362, ~: 901062)
[PASS] test_RevertWhen_redeemWithoutAllowance((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 804361,
  ↳ ~: 806584)
[PASS] test_RevertWhen_withdrawWithoutAllowance((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, :
  ↳ 823722, ~: 826532)
```

```
[PASS] test_asset((address[4],uint256[4],uint256[4],int256)) (runs: 256, : 756640, ~: 759527)
[PASS] test_convertToAssets((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 765488, ~: 767922)
[PASS] test_convertToShares((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 766411, ~: 768762)
[PASS] test_deposit((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, : 851004, ~: 854546)
[PASS] test_maxDeposit((address[4],uint256[4],uint256[4],int256)) (runs: 256, : 756749, ~: 759636)
[PASS] test_maxMint((address[4],uint256[4],uint256[4],int256)) (runs: 256, : 758157, ~: 761044)
[PASS] test_maxRedeem((address[4],uint256[4],uint256[4],int256)) (runs: 256, : 757901, ~: 760788)
[PASS] test_maxWithdraw((address[4],uint256[4],uint256[4],int256)) (runs: 256, : 764210, ~: 766963)
[PASS] test_mint((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, : 855688, ~: 858311)
[PASS] test_previewDeposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 848493, ~: 851695)
[PASS] test_previewMint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 852504, ~: 855633)
[PASS] test_previewRedeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, : 849228, ~: 851379)
[PASS] test_previewWithdraw((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, : 861812, ~: 864260)
[PASS] test_redeem((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, : 853729, ~: 855559)
[PASS] test_totalAssets((address[4],uint256[4],uint256[4],int256)) (runs: 256, : 759040, ~: 761927)
[PASS] test_withdraw((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, : 864109, ~: 867354)
Suite result: ok. 26 passed; 0 failed; 0 skipped; finished in 47.20s (16.49s CPU time)
```

Ran 5 tests for test/invariants/StatefulMarketInvariant.t.sol:StatefulMarketInvariantTest

```
[PASS] invariant_flowConservation() (runs: 256, calls: 128000, reverts: 140)
```

Contract	Selector	Calls	Reverts	Discards
MarketHandler	deposit	15889	0	0
MarketHandler	movePrice	16073	0	0
MarketHandler	redeem	15782	0	0
MarketHandler	setFees	15791	0	0
MarketHandler	sweepFees	16117	0	0
MarketHandler	touch	16175	0	0
MarketHandler	warp	16187	0	0
MarketHandler	withdraw	15986	140	0

```
[PASS] invariant_internalAccountingMatchesBalance() (runs: 256, calls: 128000, reverts: 143)
```

Contract	Selector	Calls	Reverts	Discards
MarketHandler	deposit	15889	0	0
MarketHandler	movePrice	16073	0	0
MarketHandler	redeem	15782	0	0
MarketHandler	setFees	15791	0	0
MarketHandler	sweepFees	16117	0	0
MarketHandler	touch	16175	0	0
MarketHandler	warp	16187	0	0
MarketHandler	withdraw	15986	143	0

```
[PASS] invariant_poolTotalAssetsMatchState() (runs: 256, calls: 128000, reverts: 145)
```

Contract	Selector	Calls	Reverts	Discards
MarketHandler	deposit	15889	0	0
MarketHandler	movePrice	16073	0	0
MarketHandler	redeem	15782	0	0
MarketHandler	setFees	15791	0	0

MarketHandler	sweepFees	16117	0	0
MarketHandler	touch	16175	0	0
MarketHandler	warp	16187	0	0
MarketHandler	withdraw	15986	145	0

[PASS] invariant_poolsNeverHoldAssets() (runs: 256, calls: 128000, reverts: 138)

Contract	Selector	Calls	Reverts	Discards
MarketHandler	deposit	15889	0	0
MarketHandler	movePrice	16073	0	0
MarketHandler	redeem	15782	0	0
MarketHandler	setFees	15791	0	0
MarketHandler	sweepFees	16117	0	0
MarketHandler	touch	16175	0	0
MarketHandler	warp	16187	0	0
MarketHandler	withdraw	15986	138	0

[PASS] invariant_sharesRemainRedeemable() (runs: 256, calls: 128000, reverts: 131)

Contract	Selector	Calls	Reverts	Discards
MarketHandler	deposit	15889	0	0
MarketHandler	movePrice	16073	0	0
MarketHandler	redeem	15782	0	0
MarketHandler	setFees	15791	0	0
MarketHandler	sweepFees	16117	0	0
MarketHandler	touch	16175	0	0
MarketHandler	warp	16187	0	0
MarketHandler	withdraw	15986	131	0

Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 47.07s (197.24s CPU time)

Ran 6 tests for test/invariants/InvariantTests.t.sol:InvariantTestsTest

[PASS] testFuzz_invariant_computeFundingRate_isAntisymmetric(uint96,uint96) (runs: 256, : 18158, ~: 18219)

[PASS] testFuzz_invariant_computeFundingRate_isBounded(uint128,uint128,uint256) (runs: 256, : 257382, ~: 257456)

[PASS] testFuzz_invariant_computeFundingRate_isMonotonic(uint128,uint128,uint128) (runs: 256, : 23068, ~: 23046)

[PASS] testFuzz_invariant_computeFundingRate_isZeroAtTargetCoverage(uint96) (runs: 256, : 14539, ~: 14553)

[PASS] testFuzz_invariant_updatePoolsForFunding_conserveAndTransfers(uint128,uint128,uint40) (runs: 256, : 27495, ~: 27296)

[PASS] testFuzz_invariant_updatePoolsForReturns_conserveAssets(uint128,uint128,uint128,uint128,uint40) (runs: 256, : 18435, ~: 18368)

Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 47.32s (164.81ms CPU time)

Ran 32 test suites in 47.33s (195.41s CPU time): 447 tests passed, 0 failed, 2 skipped (449 total tests)

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.